

Guia de estudo: teste do Hotel Management System

Tudo o que foi feito, caso a caso, para estudar antes da apresentação. Os números vêm das mesmas evidências do relatório.

1. Resumo em um minuto

Testei um sistema web de reservas de hotel (Flask) em três requisitos da reserva: **REQ-01 Reservar quartos**, **REQ-02 Data de entrada** e **REQ-03 Data de saída**. São **15 casos de teste**, os mesmos nas três etapas: a estrutural reaproveitou 5 deles e ampliou 3; a de mutação rodou os 15 e ampliou 4. Nenhum caso novo foi criado depois da etapa funcional. Eles revelaram **10 defeitos** no recorte, todos corrigidos no fork. Cobertura final: 100% dos comandos e dos desvios. Mutação: 127/141 (90,1%) → 134/141 (95,0%), e 100% sem os 7 equivalentes.

Etapa	Código	Casos (testes pytest)	Passaram	Falharam (defeito)	Comandos	Desvios	Mutação
1. Funcional	original	15 (15)	6	9	60/63 (95,2%)	29/34 (85,3%)	—
2. Estrutural	original	15 (18)	5	10	63/63 (100%)	33/34 (97,1%)	—
3. Correção	corrigida	15 (18)	15	0	65/65 (100%)	28/28 (100%)	127/141
4. Mutação	corrigida	15 (22)	15	0	65/65 (100%)	28/28 (100%)	134/141

2. Conceitos que você precisa saber

- **Classe de equivalência (CE):** Grupo de entradas que o sistema deveria tratar do mesmo jeito. Se um valor do grupo funciona, os outros também deveriam. Ex.: "hóspedes de 1 até a capacidade" é uma classe válida; "menos de 1 hóspede" é uma classe inválida. Testa-se um representante de cada classe.
- **Análise do valor limite (AVL):** Os erros se concentram nas fronteiras. Por isso o teste usa o valor exatamente no limite e logo fora dele: 0 e 1 hóspede, ontem e hoje, 0 e 1 noite, 5 e 6 hóspedes.
- **Caso de teste:** Uma situação concreta com entrada, ação e resultado esperado. Aqui, cada caso é uma função pytest chamada `test_CT_xxx_...`
- **Assertiva (assert):** A linha que diz o que precisa ser verdade para o teste passar. Se qualquer assert for falso, o teste falha.
- **xfail estrito:** Marca usada no código ORIGINAL nos testes que revelam defeito: o teste precisa falhar. Se um dia ele passar, o pytest acusa erro, porque o defeito "sumiu" sem explicação. No código corrigido não há marca e o teste precisa passar.
- **Cobertura de comandos:** Porcentagem das linhas de código que pelo menos um teste executou.
- **Cobertura de desvios:** Porcentagem dos caminhos de cada if/for (lado verdadeiro e lado falso) que algum teste percorreu. É mais exigente que a de comandos.
- **Desvio inviável:** Caminho que nenhuma entrada consegue percorrer. Aqui, o 230→229 do original: os IDs das reservas vêm em ordem crescente, então a condição "`rid > maior anterior`" nunca é falsa.
- **Grafo de fluxo de controle:** Desenho da função em nós (comandos e decisões) e arestas (caminhos possíveis). O V&V TestLab pinta de verde o que cada teste executou.
- **Mutante:** Cópia do código com UMA alteração pequena, como trocar `<` por `<=`. Simula um erro de programação.
- **Mutante morto / sobrevivente:** Morto: algum teste falhou com a alteração, ou seja, a suíte percebeu o erro. Sobrevivente: todos os testes passaram, ou seja, a suíte deixaria esse erro passar.
- **Mutante equivalente:** Alteração que não muda o comportamento do programa (ex.: um valor que é sobrescrito antes de ser usado). Nenhum teste consegue matá-lo; por isso é descontado do escore ajustado.

- **Escore de mutação:** Mortos ÷ total de mutantes. Mede a força da suíte: quanto maior, mais erros ela detectaria.

3. Como ler o código dos testes

- Cada teste começa com um banco SQLite vazio, criado na memória pela fixture baseline: quartos 101 (R\$ 100 por noite, cabem 2 pessoas), 102 (R\$ 150, 3 pessoas) e 301 (R\$ 200, 4 pessoas), e dois usuários, Ana e Bruno.
- As datas são reais (março de 2030), escritas como no Brasil (DD/MM/AAAA). A exceção são os casos de 'hoje' e 'ontem', que usam hoje() e mudam conforme o dia em que o teste roda.
- login(client) coloca na sessão que a Ana está logada, como se ela tivesse feito login.
- reservar(client, quartos, hospedes, entrada, saída) preenche e envia o formulário de reserva (POST /reserve). O que não for informado usa o padrão: quarto 101, 2 hóspedes, de 10/03/2030 a 12/03/2030.
- reserva_existente(...) grava direto no banco uma reserva que já existia antes do teste (por padrão, quarto 101 de 10/03/2030 a 12/03/2030).
- path(resposta) diz para onde o sistema mandou o usuário: /rooms significa que a reserva foi ACEITA; /reserve significa que foi RECUSADA (volta ao formulário com mensagem); / é a página inicial (usuário sem login).
- Uma estadia é o intervalo [entrada, saída): a noite da saída não conta. Por isso uma pessoa pode entrar no mesmo dia em que outra sai.
- Os mesmos 15 casos são usados nas três etapas. Quando uma etapa precisou de um cenário a mais, o caso foi AMPLIADO: a ampliação é outra função pytest com o mesmo número de caso (por exemplo, test_CT_001_ampliacao_...).

Exemplo anotado (CT-009):

```
@funcional                                # etapa em que o caso foi criado
@pytest.mark.ce("CE-12")                   # classe exercitada: menos de 1 hóspede
@pytest.mark defeito("DEF-03", ...)        # no original, precisa falhar (xfail estrito)
def test_CT_009_reserva_sem_hospedes_e_recusada(client):
    login(client)                          # Ana logada
    assert path(reservar(client, hospedes=0)) == "/reserve" # reserva com 0 hóspedes deve ser RECUSADA
    assert Reservations.query.count() == 0          # e nada pode ter sido gravado
```

4. Etapa 1: teste funcional

1. Li só o README e o formulário de reserva, sem olhar o código (caixa-preta), e listei as condições de entrada: sessão, número dos quartos (formato, existência, repetição, quantidade), número de hóspedes, ocupação do quarto no período, data de entrada (em relação a hoje e formato) e duração da estadia.
2. Dividi cada condição em classes de equivalência válidas e inválidas: 22 classes no total, 11 válidas e 11 inválidas (tabela em Requisitos e no relatório, seção 4.1).
3. Um único caso válido (CT-001) cobre todas as classes válidas de uma vez. Cada classe inválida ganhou um caso próprio, em que todo o resto é válido, para que a recusa só possa ter uma causa. Resultado: 15 casos.
4. Análise do valor limite: cada caso usa o valor exatamente na fronteira ou logo fora dela: 0 e 1 hóspede, 5 e 6 hóspedes (capacidade somada do 101 e 102), ontem e hoje, 0 e 1 noite, uma noite de sobreposição (11/03) e estadias encostadas (entrar em 12/03, dia em que a outra sai).
5. Rodei os 15 casos no código ORIGINAL. Cada caso que espera revelar um defeito é marcado como xfail estrito: ele PRECISA falhar no original. Resultado: 6 passaram e 9 falharam, revelando 9 defeitos.
6. Medi a cobertura com coverage.py: 95,2% dos comandos e 85,3% dos desvios das funções reserve e cal_cost. Esse é o ponto de partida da etapa estrutural.

As 22 classes de equivalência:

Classe	Requisito	Condição	Tipo	Descrição
CE-01	REQ-01	C01 Sessão	válida	Usuário autenticado
CE-02	REQ-01	C01 Sessão	inválida	Sem sessão (nunca fez login) ou sessão encerrada
CE-03	REQ-01	C02 Formato dos quartos	válida	Números inteiros separados por vírgula
CE-04	REQ-01	C02 Formato dos quartos	inválida	Texto não numérico
CE-05	REQ-01	C03 Existência dos quartos	válida	Todos os quartos existem
CE-06	REQ-01	C03 Existência dos quartos	inválida	Algum quarto não existe
CE-07	REQ-01	C04 Repetição de quartos	válida	Sem repetição
CE-08	REQ-01	C04 Repetição de quartos	inválida	Quarto repetido
CE-09	REQ-01	C05 Quantidade de quartos	válida	Um quarto
CE-10	REQ-01	C05 Quantidade de quartos	válida	Vários quartos (custo e capacidade somados)
CE-11	REQ-01	C06 Nº de hóspedes	válida	Inteiro de 1 até a capacidade somada dos quartos
CE-12	REQ-01	C06 Nº de hóspedes	inválida	Menor que 1
CE-13	REQ-01	C06 Nº de hóspedes	inválida	Maior que a capacidade somada
CE-14	REQ-01	C06 Nº de hóspedes	inválida	Não inteiro (texto)
CE-15	REQ-01	C07 Ocupação do quarto no período	válida	Nenhuma estadia do quarto com interseção (inclui estadias adjacentes e reservas de outros quartos)
CE-16	REQ-01	C07 Ocupação do quarto no período	inválida	Estadia existente do quarto com interseção de ao menos uma noite
CE-17	REQ-02	C08 Entrada em relação a hoje	válida	Hoje ou data futura
CE-18	REQ-02	C08 Entrada em relação a hoje	inválida	Data passada
CE-19	REQ-02	C09 Formato das datas	válida	Entrada e saída válidas no formato MM/DD/AAAA
CE-20	REQ-02	C09 Formato das datas	inválida	Data malformada, inexistente ou ausente
CE-21	REQ-03	C10 Duração da reserva (saída – entrada)	válida	1 noite ou mais; cada noite é uma diária cobrada
CE-22	REQ-03	C10 Duração da reserva (saída – entrada)	inválida	0 noites ou negativa (saída igual ou anterior à entrada)

CT-001 · Reserva simples de um quarto é aceita

AVL · REQ-01 · classes CE-01, CE-03, CE-05, CE-07, CE-09, CE-11, CE-15, CE-17, CE-19, CE-21

Em uma frase: Mostra que o caminho feliz funciona e calcula o custo certo; serve de referência para todos os casos inválidos.

Por que existe. É o caso válido de base. Uma única reserva correta cobre de uma vez as 10 classes válidas: usuário logado, quarto em formato numérico, quarto existente, sem repetição, um quarto, hóspedes dentro da capacidade, quarto livre, entrada no futuro, datas no formato certo e pelo menos 1 noite. Também é o ponto de valor limite '1 hóspede' (o mínimo permitido).

Cenário, passo a passo:

1. Ana está logada.
2. Ela reserva o quarto 101 para 1 hóspede, de 10/03/2030 a 12/03/2030 (2 noites).

O que o teste confere. O sistema aceita (vai para /rooms), grava uma reserva com 1 hóspede e custo R\$ 200 (2 noites × R\$ 100) e cria um único vínculo entre a reserva e o quarto 101.

No código original. Passou: o sistema original já fazia a reserva simples corretamente.

No código corrigido. Passou.

Código (tests/test_01_funcional.py, linha 36):

```
@funcional
@pytest.mark.ce("CE-01", "CE-03", "CE-05", "CE-07", "CE-09", "CE-11", "CE-15", "CE-17", "CE-19", "CE-21")
def test_CT_001_reserva_simples_de_um_quarto_e_aceita(client):
    login(client)
    resposta = reservar(client, quartos="101", hospedes=1, entrada="10/03/2030", saida="12/03/2030")
    assert path(resposta) == "/rooms"
    reserva = Reservations.query.one()
    assert reserva.num_guests == 1
    assert reserva.costs == 200 # 2 noites x R$ 100
    assert [(b.brid, b.room_id) for b in Booked.query.all()] == [(reserva.rid, 101)]
```

CT-002 · Reserva de dois quartos com lotação máxima é aceita

AVL · REQ-01 · classes CE-10, CE-11, CE-05

Em uma frase: Testa o limite máximo de hóspedes quando a capacidade é a soma de vários quartos.

Por que existe. Classe 'vários quartos' (CE-10) e valor limite superior de hóspedes: 101 + 102 comportam 2 + 3 = 5 pessoas, e o caso usa exatamente 5.

Cenário, passo a passo:

1. Ana está logada.
2. Ela reserva os quartos 101 e 102 juntos, para 5 hóspedes, de 10/03/2030 a 13/03/2030 (3 noites).

O que o teste confere. O sistema aceita, grava o custo (R\$ 100 + R\$ 150) × 3 = R\$ 750 e cria vínculos com os dois quartos.

No código original. Passou: o original soma a capacidade e o custo dos quartos corretamente.

No código corrigido. Passou.

Código (tests/test_01_funcional.py, linha 48):

```
@funcional
@pytest.mark.ce("CE-10", "CE-11")
def test_CT_002_reserva_de_dois_quartos_com_lotacao_maxima_e_aceita(client):
    login(client)
    resposta = reservar(client, quartos="101,102", hospedes=5, entrada="10/03/2030", saida="13/03/2030")
    assert path(resposta) == "/rooms"
    assert Reservations.query.one().costs == (100 + 150) * 3
    assert sorted(b.room_id for b in Booked.query.all()) == [101, 102]
```

CT-003 · Estadia que começa quando outra termina é aceita

AVL · REQ-01 · classes CE-15, CE-17, CE-21 · revela DEF-01

Em uma frase: O limite 'entrar no dia da saída' revelou o defeito mais grave do sistema: um quarto reservado uma vez nunca mais podia ser reservado.

Por que existe. Valor limite da ocupação: a nova estadia começa exatamente no dia em que a outra termina. Como a noite da saída não conta, as duas não se sobrepõem e a reserva deve ser aceita.

Cenário, passo a passo:

1. Já existe uma reserva do quarto 101 de 10/03/2030 a 12/03/2030.
2. Ana tenta reservar o mesmo quarto 101 de 12/03/2030 a 14/03/2030 (entra no dia em que a outra pessoa sai).

O que o teste confere. O sistema aceita e passam a existir 2 reservas.

No código original. FALHOU: o original recusou com 'The room you are reserving is not available'. Causa (DEF-01): a condição de conflito de datas é uma tautologia, com 4 alternativas ligadas por 'or' que cobrem todas as combinações possíveis de datas; ela é SEMPRE verdadeira. Depois da primeira reserva, o quarto fica bloqueado para sempre.

No código corrigido. Passou depois do commit 2d1b2cd, que trocou a condição por `_periodos_conflitam(a1, a2, b1, b2) = a1 < b2 and b1 < a2` (há conflito só se os períodos se cruzam).

Código (`tests/test_01_funcional.py`, linha 58):

```
@funcional
@pytest.mark.ce("CE-15", "CE-17")
@pytest.mark.defeito("DEF-01", "períodos sem interseção são tratados como conflito")
def test_CT_003_estadia_que_comeca_quando_outra_termina_e_aceita(client, baseline):
    reserva_existente(baseline["ana"], entrada="10/03/2030", saida="12/03/2030")
    login(client)
    assert path(reservar(client, entrada="12/03/2030", saida="14/03/2030")) == "/rooms"
    assert Reservations.query.count() == 2
```

CT-004 · Entrada hoje é aceita

AVL · REQ-02 · classes CE-17, CE-21 · revela DEF-02

Em uma frase: O limite 'hoje' mostrou um erro clássico de comparar data com data e hora.

Por que existe. Valor limite da data de entrada (o mínimo permitido é hoje) e da duração (o mínimo é 1 noite). Por depender do dia, este caso usa a data de hoje, e não uma data fixa.

Cenário, passo a passo:

1. Ana está logada.
2. Ela reserva o quarto 101 entrando HOJE e saindo amanhã (1 noite).

O que o teste confere. O sistema aceita e grava o custo de exatamente uma diária: R\$ 100.

No código original. FALHOU: o original recusou com 'Please recheck your date! It has to be at least today!'. Causa (DEF-02): a data de entrada vira 'hoje à meia-noite' e é comparada com 'hoje agora'. Meia-noite é sempre antes de agora, então o dia de hoje é tratado como passado.

No código corrigido. Passou depois do commit 704bc86, que compara só as datas, sem as horas.

Código (tests/test_01_funcional.py, linha 68):

```
@funcional
@pytest.mark.ce("CE-17", "CE-21")
@pytest.mark.defeito("DEF-02", "entrada no dia atual é rejeitada")
def test_CT_004_entrada_hoje_e_aceita(client):
    login(client)
    assert path(reservar(client, entrada=hoje(), saida=hoje(1))) == "/rooms"
    assert Reservations.query.one().costs == 100 # 1 diária
```

CT-005 · Visitante sem login não reserva

CE · REQ-01 · classes CE-02 · revela DEF-07

Em uma frase: Um visitante sem login derrubava a página com erro 500.

Por que existe. Classe inválida CE-02: visitante sem login.

Cenário, passo a passo:

1. Ninguém está logado (a sessão nem tem a informação de login).
2. O visitante abre o formulário de reserva e depois tenta enviar uma reserva.

O que o teste confere. Nas duas tentativas o sistema manda para a página inicial (/) e nenhuma reserva é gravada.

No código original. FALHOU com erro interno 500: KeyError: 'user_available'. Causa (DEF-07): o código lê session['user_available'], mas essa informação só existe depois que alguém faz login ou logout. Para um visitante novo, o sistema quebra em vez de redirecionar.

No código corrigido. Passou depois do commit 8a6c4bd, que passou a ler a sessão com segurança (session.get).

Código (tests/test_01_funcional.py, linha 79):

```
@funcional
@pytest.mark.ce("CE-02")
@pytest.mark.defeito("DEF-07", "rota protegida sem sessão gera erro interno")
def test_CT_005_visitante_sem_login_nao_reserva(client):
    assert path(client.get("/reserve")) == "/"
    assert path(reservar(client)) == "/"
    assert Reservations.query.count() == 0
```

CT-006 · Quarto escrito em texto é recusado

CE · REQ-01 · classes CE-04 · revela DEF-04

Em uma frase: Digitar texto no campo de quartos derrubava o sistema.

Por que existe. Classe inválida CE-04: número de quarto que não é número.

Cenário, passo a passo:

1. Ana está logada.
2. No campo de quartos ela digita 'abc' (o resto do formulário está correto: 2 hóspedes, de 10/03/2030 a 12/03/2030).

O que o teste confere. O sistema recusa (volta para /reserve) e não grava nada.

No código original. FALHOU com erro interno 500: ValueError: invalid literal for int() with base 10: 'abc' (linha 215). Causa (DEF-04): o código transforma o texto em número sem tratar o erro.

No código corrigido. Passou depois do commit 644fd0d: a função _ler_quartos trata o erro e a reserva é recusada com mensagem.

Código (tests/test_01_funcional.py, linha 88):

```
@funcional
@pytest.mark.ce("CE-04")
@pytest.mark.defeito("DEF-04", "número de quarto não numérico gera erro interno")
def test_CT_006_quarto_escrito_em_texto_e_recusado(client):
    login(client)
    assert path(reservar(client, quartos="abc")) == "/reserve"
    assert Reservations.query.count() == 0
```

CT-007 · Quarto que não existe é recusado

CE · REQ-01 · classes CE-06 · revela DEF-05

Em uma frase: Era possível reservar um quarto que não existe.

Por que existe. Classe inválida CE-06: quarto que não existe. O caso mistura um quarto válido (101) com um inexistente (999), mantendo 2 hóspedes, para que a ÚNICA coisa errada seja o quarto 999.

Cenário, passo a passo:

1. Ana está logada.
2. Ela pede os quartos '101,999' para 2 hóspedes, de 10/03/2030 a 12/03/2030.

O que o teste confere. O sistema recusa, não grava a reserva e não cria nenhum vínculo de quarto.

No código original. FALHOU: o original ACEITOU a reserva e criou um vínculo com o quarto 999, que não existe. Causa (DEF-05): o código nunca verifica se o quarto existe.

No código corrigido. Passou depois do commit 644fd0d: só são aceitos quartos que existem no hotel.

Código (tests/test_01_funcional.py, linha 97):

```
@funcional
@pytest.mark.ce("CE-06")
@pytest.mark.defeito("DEF-05", "quarto inexistente é aceito junto com um quarto válido")
def test_CT_007_quarto_que_nao_existe_e_recusado(client):
    login(client)
    assert path(reservar(client, quartos="101,999", hospedes=2)) == "/reserve"
    assert Reservations.query.count() == 0
    assert Booked.query.count() == 0
```

CT-008 · Quarto repetido é recusado

CE · REQ-01 · classes CE-08 · revela DEF-06

Em uma frase: Repetir o quarto dobrava a capacidade e o preço.

Por que existe. Classe inválida CE-08: quarto repetido na lista.

Cenário, passo a passo:

1. Ana está logada.
2. Ela pede os quartos '101,101' para 2 hóspedes, de 10/03/2030 a 12/03/2030.

O que o teste confere. O sistema recusa e não grava nada.

No código original. FALHOU: o original aceitou, criou 2 vínculos com o mesmo quarto, contou capacidade 4 (2 + 2) e cobrou R\$ 400 em vez de R\$ 200. Causa (DEF-06): a lista de quartos não é validada.

No código corrigido. Passou depois do commit 644fd0d: listas com quartos repetidos são recusadas.

Código (tests/test_01_funcional.py, linha 107):

```
@funcional
@pytest.mark.ce("CE-08")
@pytest.mark.defeito("DEF-06", "quarto repetido gera dois vínculos e custo em dobro")
def test_CT_008_quarto_repetido_e_recusado(client):
    login(client)
    assert path(reservar(client, quartos="101,101", hospedes=2)) == "/reserve"
    assert Reservations.query.count() == 0
```

CT-009 · Reserva sem hóspedes é recusada

AVL · REQ-01 · classes CE-12 · revela DEF-03

Em uma frase: O limite '0 hóspedes' mostrou que faltava a validação do mínimo.

Por que existe. Classe inválida CE-12 (menos de 1 hóspede) e valor limite logo abaixo do mínimo: 0 hóspedes.

Cenário, passo a passo:

1. Ana está logada.
2. Ela reserva o 101 para 0 hóspedes, de 10/03/2030 a 12/03/2030.

O que o teste confere. O sistema recusa e não grava nenhuma reserva.

No código original. FALHOU: o original aceitou e gravou uma reserva com 0 hóspedes e custo R\$ 200. Causa (DEF-03): o código só verifica o máximo de hóspedes, nunca o mínimo.

No código corrigido. Passou depois do commit 644fd0d, que exige pelo menos 1 hóspede antes de gravar.

Código (tests/test_01_funcional.py, linha 116):

```
@funcional
@pytest.mark.ce("CE-12")
@pytest.mark defeito("DEF-03", "reserva com zero hóspedes é gravada")
def test_CT009_reserva_sem_hospedes_e_recusada(client):
    login(client)
    assert path(reservar(client, hospedes=0)) == "/reserve"
    assert Reservations.query.count() == 0
```

CT-010 · Hóspedes acima da capacidade são recusados

AVL · REQ-01 · classes CE-13

Em uma frase: Junto com o CT-002, confirma a fronteira 5 aceita / 6 recusa.

Por que existe. Classe inválida CE-13 (mais hóspedes que a capacidade) e valor limite logo acima do máximo: 101 + 102 comportam 5, o caso pede 6.

Cenário, passo a passo:

1. Ana está logada.
2. Ela reserva 101 e 102 para 6 hóspedes, de 10/03/2030 a 12/03/2030.

O que o teste confere. O sistema recusa e não grava nada.

No código original. Passou: o original já recusava com 'The rooms you selected can't fit the number of guests'.

No código corrigido. Passou.

Código (tests/test_01_funcional.py, linha 125):

```
@funcional
@pytest.mark.ce("CE-13")
def test_CT010_hospedes_acima_da_capacidade_sao_recusados(client):
    login(client)
    assert path(reservar(client, quartos="101,102", hospedes=6)) == "/reserve"
    assert Reservations.query.count() == 0
```

CT-011 · Hóspedes escritos por extenso são recusados

CE · REQ-01 · classes CE-14 · revela DEF-19

Em uma frase: Escrever o número por extenso derrubava o sistema.

Por que existe. Classe inválida CE-14: número de hóspedes que não é inteiro.

Cenário, passo a passo:

1. Ana está logada.
2. No campo de hóspedes ela digita 'dois' (reserva do 101 de 10/03/2030 a 12/03/2030).

O que o teste confere. O sistema recusa e não grava nada.

No código original. FALHOU com erro interno 500: `TypeError: '>' not supported between instances of 'NoneType' and 'int'` (linha 217). Causa (DEF-19): o campo inválido fica vazio (None) e o código o compara com a capacidade sem validar.

No código corrigido. Passou depois do commit 644fd0d, que recusa hóspedes ausentes ou inválidos antes de qualquer conta.

Código (`tests/test_01_funcional.py`, linha 133):

```
@funcional
@pytest.mark.ce("CE-14")
@pytest.mark defeito("DEF-19", "número de hóspedes não inteiro gera erro interno")
def test_CT_011_hospedes_escritos_por_extenso_sao_recusados(client):
    login(client)
    assert path(reservar(client, hospedes="dois")) == "/reserve"
    assert Reservations.query.count() == 0
```

CT-012 · Quarto ocupado no período é recusado

AVL · REQ-01 · classes CE-16, CE-15 · revela DEF-15

Em uma frase: Garante que a correção do DEF-01 não deixou passar sobreposições reais.

Por que existe. Classe inválida CE-16 (quarto ocupado no período) e valor limite da sobreposição: só UMA noite em comum.

Cenário, passo a passo:

1. Já existe uma reserva do 101 de 10/03/2030 a 12/03/2030.
2. Ana tenta reservar o 101 de 11/03/2030 a 13/03/2030 (a noite de 11/03 está nas duas).

O que o teste confere. O sistema recusa e continua existindo só 1 reserva.

No código original. Passou, mas por acaso: no original a regra de conflito é sempre verdadeira (DEF-01), então ele recusa QUALQUER reserva de um quarto que já tem reserva. Acertou o resultado pelo motivo errado; quem mostra o defeito é o CT-003.

No código corrigido. Passou, agora pelo motivo certo: `_periodos_conflitam` detecta a noite em comum.

Código (`tests/test_01_funcional.py`, linha 142):

```
@funcional
@pytest.mark.ce("CE-16")
def test_CT_012_quarto_ocupado_no_periodo_e_recusado(client, baseline):
    reserva_existente(baseline["ana"], entrada="10/03/2030", saida="12/03/2030")
    login(client)
    assert path(reservar(client, entrada="11/03/2030", saida="13/03/2030")) == "/reserve" # noite de
11/03 em comum
    assert Reservations.query.count() == 1
```

CT-013 · Entrada no passado é recusada

AVL · REQ-02 · classes CE-18

Em uma frase: Junto com o CT-004, confirma a fronteira ontem recusa / hoje aceita.

Por que existe. Classe inválida CE-18 (entrada no passado) e valor limite logo abaixo do mínimo: ontem. Por depender do dia, usa a data de ontem, e não uma data fixa.

Cenário, passo a passo:

1. Ana está logada.
2. Ela tenta reservar com entrada ONTEM e saída amanhã.

O que o teste confere. O sistema recusa e não grava nada.

No código original. Passou: o original já recusava datas passadas.

No código corrigido. Passou.

Código (`tests/test_01_funcional.py`, linha 151):

```
@funcional
@pytest.mark.ce("CE-18")
def test_CT_013_entrada_no_passado_e_recusada(client):
    login(client)
    assert path(reservar(client, entrada=hoje(-1), saida=hoje(1))) == "/reserve" # entrada ontem
    assert Reservations.query.count() == 0
```

CT-014 · Data em formato errado é recusada

CE · REQ-02 · classes CE-20 · revela DEF-18

Em uma frase: Uma data mal digitada derrubava o sistema.

Por que existe. Classe inválida CE-20: data fora do formato MM/DD/AAAA.

Cenário, passo a passo:

1. Ana está logada.
2. Ela envia a data de entrada '2030-13-01' (formato errado e mês 13); a saída e o resto estão corretos.

O que o teste confere. O sistema recusa e não grava nada.

No código original. FALHOU com erro interno 500: TypeError: combine() argument 1 must be datetime.date, not None (linha 191). Causa (DEF-18): o campo de data inválido fica vazio (None) e o código o usa sem verificar.

No código corrigido. Passou depois do commit 644fd0d, que recusa datas ausentes ou inválidas antes de usá-las.

Código (tests/test_01_funcional.py, linha 159):

```
@funcional
@pytest.mark.ce("CE-20")
@pytest.mark defeito("DEF-18", "data fora do formato MM/DD/AAAA gera erro interno")
def test_CT_014_data_em_formato_errado_e_recusada(client):
    login(client)
    resposta = client.post("/reserve", data={"checkin_date": "2030-13-01", "checkout_date": "03/12/2030",
                                             "num_guests": "2", "room_numbers": "101"})

    assert path(resposta) == "/reserve"
    assert Reservations.query.count() == 0
```

CT-015 · Saída igual à entrada é recusada

AVL · REQ-03 · classes CE-22

Em uma frase: Junto com o CT-004, confirma a fronteira 0 noites recusa / 1 noite aceita.

Por que existe. Classe inválida CE-22 (saída igual ou anterior à entrada) e valor limite da duração: 0 noites.

Cenário, passo a passo:

1. Ana está logada.
2. Ela tenta reservar com entrada e saída em 10/03/2030.

O que o teste confere. O sistema recusa e não grava nada.

No código original. Passou: o original já recusava (saída <= entrada).

No código corrigido. Passou.

Código (tests/test_01_funcional.py, linha 170):

```
@funcional
@pytest.mark.ce("CE-22")
def test_CT_015_saida_igual_a_entrada_e_recusada(client):
    login(client)
    assert path(reservar(client, entrada="10/03/2030", saida="10/03/2030")) == "/reserve" # 0 noites
    assert Reservations.query.count() == 0
```

5. Etapa 2: teste estrutural

1. Não criei casos novos: reaproveitei 5 dos 15 casos funcionais, um para cada caminho principal da função reserve. CT-001 (reserva aceita), CT-005 (sem login), CT-009 (dado inválido), CT-012 (quarto ocupado) e CT-013 (data no passado). O grafo de cada função mostra em verde o caminho que cada um percorre.

2. Rodei os 15 casos com `coverage.py --branch` e li o relatório `term-missing`, que lista as linhas e os desvios que nenhum teste executou: linha 241 (abrir o formulário com GET), linhas 242-243 (usuário que saiu do sistema), desvios 208→205 e 205→204 (reserva já existente de OUTRO quarto) e 230→229.
3. Para cada lacuna, AMPLIEI um dos casos reaproveitados com o cenário que faltava: o CT-001 passou a abrir o formulário antes de reservar, com outro quarto já ocupado nas mesmas datas; o CT-005 passou a testar também o usuário que fez logout.
4. Ao ler o laço triplo para entender o desvio 208→205, percebi que ele compara cada quarto reservado com as datas de TODAS as reservas. Ampliei o CT-012 com o cenário que prova isso, e ele revelou o DEF-15.
5. O desvio 230→229 é inviável: os números das reservas vêm em ordem crescente, então a condição nunca é falsa.
6. Resultado no original: os mesmos 15 casos (18 funções pytest), 100% dos comandos e 33 de 34 desvios (o que falta é o inviável). Meta atingida.

Os 5 casos reaproveitados e o caminho que cada um percorre no grafo de reserve:

Caso	Nome	Caminho no grafo	Ampliado?
CT-001	Reserva simples de um quarto é aceita	Caminho da reserva aceita: usuário logado → formulário enviado → dados válidos → datas válidas → quarto livre → hóspedes cabem → grava → vai para /rooms.	sim
CT-005	Visitante sem login não reserva	Caminho do visitante sem login: o primeiro teste (usuário logado?) dá 'Não' → aviso → volta à página inicial.	sim
CT-009	Reserva sem hóspedes é recusada	Caminho do dado inválido: o teste 'Algum dado inválido?' dá 'Sim' → aviso → volta ao formulário.	não
CT-012	Quarto ocupado no período é recusado	Caminho do quarto ocupado: 'Este quarto está ocupado?' dá 'Sim' → aviso → volta ao formulário.	sim
CT-013	Entrada no passado é recusada	Caminho da data no passado: 'Datas inválidas?' dá 'Sim' → aviso → volta ao formulário.	não

CT-001 ampliado na etapa estrutural: Abre o formulário antes e há outro quarto ocupado nas mesmas datas

Caso original: Reserva simples de um quarto é aceita

Por que foi ampliado. A cobertura mostrou dois trechos que nenhum caso executava: a linha 241 (mostrar o formulário quando a pessoa só abre a página) e os desvios 208→205 e 205→204 (quando já existe reserva, mas de OUTRO quarto).

Cenário acrescentado:

1. Bruno já tem o quarto 102 reservado de 10/03/2030 a 12/03/2030.
2. Ana abre a página de reserva (só abre, sem enviar).
3. Ana reserva o quarto 101 nas mesmas datas, 10/03/2030 a 12/03/2030.

O que confere. O formulário aparece normalmente e a reserva é aceita (quartos diferentes não conflitam); passam a existir 2 reservas.

No código original. Passou no original: aqui não há defeito, a ampliação existe para executar os trechos que faltavam.

Código (`tests/test_02_estrutural.py`, linha 23):

```
@estrutural
@pytest.mark.ce("CE-01", "CE-15")
def test_CT_001_ampliacao_formulario_aberto_antes_e_outro_quarto_ocupado(client, baseline):
    # Original: desvio 189->241 e linha 241 (abrir o formulário com GET, sem enviar) e
    # desvios 208->205 e 205->204 (reserva já existente de OUTRO quarto no mesmo período).
    reserva_existente(baseline["bruno"], quartos=(102,), entrada="10/03/2030", saida="12/03/2030")
    login(client)
    formulario = client.get("/reserve")
    assert formulario.status_code == 200
    assert b'name="room_numbers"' in formulario.data
    assert path(reservar(client, quartos="101", entrada="10/03/2030", saida="12/03/2030")) == "/rooms"
    assert Reservations.query.count() == 2
```

CT-005 ampliado na etapa estrutural: Usuário que saiu do sistema (logout) também não reserva

Caso original: Visitante sem login não reserva

Por que foi ampliado. A cobertura mostrou que o ramo 'usuário não autenticado' (linhas 242-243) nunca rodava: sem a informação de login o original quebra antes de chegar nele. Esse ramo só roda quando a informação existe e vale 'falso', que é o estado depois do logout. É o segundo representante da mesma classe CE-02.

Cenário acrescentado:

1. A sessão fica como depois de um logout (user_available = falso).
2. O usuário abre e tenta enviar o formulário de reserva.

O que confere. As duas tentativas voltam para a página inicial e nada é gravado.

No código original. Passou no original: a ampliação existe para executar o ramo que faltava.

Código (tests/test_02_estrutural.py, linha 37):

```
@estrutural
@pytest.mark.ce("CE-02")
def test_CT_005_ampliacao_usuario_que_saiu_nao_reserva(client):
    # Original: desvio 186->242 e linhas 242-243 (ramo falso de "if session['user_available']").
    # Sem a chave na sessão o original quebra antes do if; o ramo falso só roda depois de um logout.
    with client.session_transaction() as sessao:
        sessao["user_available"] = False
    assert path(client.get("/reserve")) == "/"
    assert path(reservar(client)) == "/"
    assert Reservations.query.count() == 0
```

CT-012 ampliado na etapa estrutural: Reserva de outro quarto não bloqueia um quarto livre (revelou o DEF-15)

Caso original: Quarto ocupado no período é recusado · revela DEF-15

Por que foi ampliado. Ao ler o laço triplo das linhas 203-212 para cobrir o desvio 208→205, ficou claro que ele compara cada quarto reservado com as datas de TODAS as reservas, de qualquer quarto (falta conferir brid == rid). Esta ampliação prova o problema.

Cenário acrescentado:

1. Ana tem o 101 reservado de 10/03/2030 a 12/03/2030.
2. Bruno tem o 102 reservado de 20/03/2030 a 22/03/2030.
3. Ana tenta reservar o 101 de 20/03/2030 a 22/03/2030 (o 101 está livre nessas datas).

O que confere. O sistema aceita e passam a existir 3 reservas.

No código original. FALHOU: recusado com 'not available'. Causa (DEF-15): o laço junta o quarto 101 (da reserva da Ana) com as datas da reserva do Bruno (20/03) e acusa um conflito que não existe. Estava MASCARADO pelo DEF-01: com a regra sempre verdadeira, tudo já era recusado.

No código corrigido. Passou depois do commit 2d1b2cd: cada quarto é comparado só com as datas da própria reserva.

Código (`tests/test_02_estrutural.py`, linha 49):

```
@estrutural
@pytest.mark.ce("CE-15")
@pytest.mark defeito("DEF-15", "vínculo de quarto é comparado com reservas de outros quartos")
def test_CT_012_ampliacao_reserva_de_outro_quarto_nao_bloqueia(client, baseline):
    # Lendo o laço triplo das linhas 203-212 para cobrir o desvio 208->205: ele combina cada
    # quarto reservado com as datas de TODAS as reservas (falta brid == rid). Aqui o 101 está
    # livre em 20/03, mas o original o recusa por causa da reserva do Bruno (quarto 102).
    reserva_existente(baseline["ana"], quartos=(101,), entrada="10/03/2030", saida="12/03/2030")
    reserva_existente(baseline["bruno"], quartos=(102,), entrada="20/03/2030", saida="22/03/2030")
    login(client)
    assert path(reservar(client, quartos="101", entrada="20/03/2030", saida="22/03/2030")) == "/rooms"
    assert Reservations.query.count() == 3
```

6. Correção dos defeitos (antes da mutação)

Cada defeito foi corrigido no fork, em commits separados. O mesmo teste que falhava no original passou a passar.

Defeito	O que era	Como foi corrigido	Caso que comprova
DEF-01	Quarto reservado uma vez nunca mais pode ser reservado	A regra passou a acusar conflito só quando os períodos realmente se cruzam (commit 2d1b2cd).	CT-003
DEF-02	Entrada no dia de hoje é recusada	Passou a comparar só as datas, sem as horas (commit 704bc86).	CT-004
DEF-03	Reserva com zero hóspedes é aceita	Agora exige pelo menos 1 hóspede (commit 644fd0d).	CT-009
DEF-04	Texto no campo de quartos derruba o sistema	Os números dos quartos são lidos com tratamento de erro; texto inválido é recusado com mensagem (commit 644fd0d).	CT-006
DEF-05	Quarto inexistente é aceito na reserva	Só aceita quartos que existem no hotel (commit 644fd0d).	CT-007
DEF-06	Quarto repetido dobra a capacidade e o preço	Recusa quartos repetidos (commit 644fd0d).	CT-008
DEF-07	Visitante sem login derruba a página de reserva	Sem login, o visitante volta para a página inicial (commit 8a6c4bd).	CT-005
DEF-18	Data fora do formato derruba o sistema	Datas inválidas são recusadas com mensagem (commits 644fd0d e 9e66e2d).	CT-014
DEF-19	Hóspedes escritos por extenso derrubam o sistema	Número de hóspedes inválido é recusado com mensagem (commit 644fd0d).	CT-011
DEF-15	Quarto livre é recusado por causa de reservas de outros quartos	Cada quarto é comparado só com as datas da própria reserva (commit 2d1b2cd).	CT-012

7. Etapa 3: teste de mutação

1. Antes de mutar, corriji os defeitos no fork (6 commits) e confirmei que os 15 casos passavam no código corrigido. A mutação precisa de uma suíte toda verde.
2. O Cosmic Ray gerou 141 mutantes das funções do recorte. Cada mutante é uma cópia do código com UMA alteração pequena, como trocar < por <=, == por is ou 0 por 1. Para cada mutante, os 15 casos rodam inteiros.
3. Se algum teste falha, o mutante MORREU: os testes perceberam que o código mudou. Se todos passam, o mutante SOBREVIVEU: existe um erro que os testes não detectariam.
4. Rodada inicial (15 casos, 18 funções): 127 mortos e 14 sobreviventes. Escore = $127 \div 141 = 90,1\%$.
5. Analisei os 14 sobreviventes um a um. 7 são EQUIVALENTES: a alteração não muda o comportamento, então nenhum teste consegue matá-los. Os outros 7 mostraram fraquezas dos casos, e em vez de criar casos novos AMPLIEI os casos que deveriam tê-los detectado: CT-002, CT-003, CT-012 e CT-015.
6. Rodada final (os mesmos 15 casos, 22 funções): 134 mortos e 7 sobreviventes, todos equivalentes. Escore = $134 \div 141 = 95,0\%$. Sem os equivalentes, $134 \div (141 - 7) = 100\%$.

Os 14 sobreviventes da rodada inicial:

#	Mutante	Onde	Alteração	Classificação	Por quê
S1	M014	_periodos_conflitam, linha 17	<code>return entrada_a < saida_b and entrada_b < saida_a → return entrada_a < saida_b and entrada_b <= saida_a</code>	Não equivalente	Nenhum caso tinha a nova estadia terminando no dia em que outra começa; a etapa funcional só testou o limite do outro lado (CT-003). Morto pela ampliação do CT-003.
S2	M012	_periodos_conflitam, linha 17	<code>return entrada_a < saida_b and entrada_b < saida_a → return entrada_a < saida_b and entrada_b is not saida_a</code>	Não equivalente	Mesmo cenário do S1: entre objetos de data distintos, 'is not' é sempre verdadeiro e acusa conflito. Morto pelo CT-020.
S3	M016	_periodos_conflitam, linha 17	<code>return entrada_a < saida_b and entrada_b < saida_a → return entrada_a < saida_b and entrada_b != saida_a</code>	Não equivalente	'!=' acusa conflito com qualquer estadia posterior não adjacente; nenhum caso reservava um período inteiramente anterior a outro. Morto pela ampliação do CT-003.
S4	M018	_quartos_ocupados, linha 23	<code>Reservations, Booked.brid == Reservations.rid) → Reservations, Booked.brid >= Reservations.rid)</code>	Não equivalente	Nenhum caso tinha um vínculo de reserva posterior capaz de herdar as datas de uma reserva anterior de outro quarto. Morto pela ampliação do CT-012 (etapa de mutação).
S5	M031	_ler_quartos, linha 34	<code>if len(set(numeros)) != len(numeros) or not set(numeros) <= existentes: → if len(set(numeros)) != len(numeros) or not set(numeros) < existentes:</code>	Não equivalente	Nenhum caso reservava todos os quartos do hotel (subconjunto próprio < em vez de <=). Morto pela ampliação do CT-002.

#	Mutante	Onde	Alteração	Classificação	Por quê
S6	M076	reserve, linha 230	<code>if d2 <= d1 or d1.date() < datetime.date.today(): → if d2 == d1 or d1.date() < datetime.date.today():</code>	Não equivalente	A classe CE-22 foi testada só com 0 noites; 'd2 == d1' deixa passar saída anterior à entrada. Morto pela ampliação do CT-015.
S7	M115	cal_cost, linha 268	<code>if each_room_id == e.room_number: → if each_room_id is e.room_number:</code>	Não equivalente	'is' só coincide com '=' para inteiros de -5 a 256 (cache do CPython); nenhum caso reservava o quarto 301. Morto pela ampliação do CT-002.
S8	M045	reserve, linha 217	<code>if request.method == 'POST': → if request.method >= 'POST':</code>	Equivalente	A rota só admite GET, POST, HEAD e OPTIONS, e nenhum método diferente de POST é >= 'POST' na ordem lexicográfica.
S9	M067	reserve, linha 226	<code>d1 = datetime.datetime.combine(checkin, datetime.time(0, 0)) → d1 = datetime.datetime.combine(checkin, datetime.time(1, 0))</code>	Equivalente	d1 só é comparado com datas à meia-noite e com date.today() via d1.date(); deslocar a hora dentro do mesmo dia não inverte nenhuma comparação.
S10	M069	reserve, linha 226	<code>d1 = datetime.datetime.combine(checkin, datetime.time(0, 0)) → d1 = datetime.datetime.combine(checkin, datetime.time(0, 1))</code>	Equivalente	Idem ao S9: mudar time(0, 0) para time(0, 1) não altera nenhuma comparação de data.
S11	M105	reserve, linha 247	<code>reserve = Reservations(us.uid, checkin, checkout, num, 0) → reserve = Reservations(us.uid, checkin, checkout, num, 1)</code>	Equivalente	O custo provisório é sobrescrito por cal_cost antes do único commit.
S12	M106	reserve, linha 247	<code>reserve = Reservations(us.uid, checkin, checkout, num, 0) → reserve = Reservations(us.uid, checkin, checkout, num, -1)</code>	Equivalente	Idem ao S11: o valor -1 é sobrescrito por cal_cost antes do commit.
S13	M038	_ler_quartos, linha 34	<code>if len(set(numeros)) != len(numeros) or not set(numeros) <= existentes: → if len(set(numeros)) < len(numeros) or not set(numeros) <= existentes:</code>	Equivalente	Um conjunto nunca tem mais elementos que a lista de origem, então != e < são a mesma condição.
S14	M037	_ler_quartos, linha 34	<code>if len(set(numeros)) != len(numeros) or not set(numeros) <= existentes: → if len(set(numeros)) is not len(numeros) or not set(numeros) <= existentes:</code>	Equivalente	Comprimentos só passam de 256 com mais de 256 quartos numa reserva (irrealista); abaixo disso 'is not' equivale a '!= '.

CT-002 ampliado na etapa de mutação: Reserva de todos os quartos do hotel, incluindo o 301

Caso original: Reserva de dois quartos com lotação máxima é aceita

Por que foi ampliado. Dois mutantes sobreviveram porque nenhum caso reservava o hotel inteiro nem o quarto 301: S5 (em `_ler_quartos`, 'todos os quartos existem' virou 'subconjunto próprio') e S7 (em `cal_cost`, '==' virou 'is', que só funciona com números até 256).

Cenário acrescentado:

1. Ana está logada.
2. Ela reserva os três quartos (101, 102 e 301) para 9 hóspedes (2 + 3 + 4), de 10/03/2030 a 12/03/2030.

O que confere. O sistema aceita, grava o custo $(100 + 150 + 200) \times 2 = \text{R\$ } 900$ e cria os 3 vínculos.

Como mata o mutante. S5: pedir todos os quartos não é subconjunto próprio, então o mutante recusa a reserva. S7: com 'is', o 301 lido do vínculo e o 301 do quarto são objetos diferentes, a diária dele não é somada e o custo fica errado. Nos dois casos o teste falha: mutantes mortos.

Código (`tests/test_03_mutacao.py`, linha 19):

```
@mutacao
@pytest.mark.ce("CE-05", "CE-10", "CE-11")
def test_CT_002_ampliacao_reserva_de_todos_os_quartos_do_hotel(client):
    # Mata _ler_quartos, ReplaceComparisonOperator_LtE_Lt: "set(numeros) < existentes" recusa
    # a reserva de TODOS os quartos. Mata também cal_cost, ReplaceComparisonOperator_Eq_Is:
    # "is" só coincide com "==" para inteiros pequenos (até 256); o quarto 301 revela a troca.
    login(client)
    assert path(reservar(client, quartos="101,102,301", hospedes=9, entrada="10/03/2030",
saida="12/03/2030")) == "/rooms"
    assert Reservations.query.one().costs == (100 + 150 + 200) * 2
    assert sorted(b.room_id for b in Booked.query.all()) == [101, 102, 301]
```

CT-003 ampliado na etapa de mutação: Estadias antes da reserva existente (o outro lado do limite)

Caso original: Estadia que começa quando outra termina é aceita

Por que foi ampliado. Três mutantes da regra de conflito sobreviveram (S1, S2 e S3) porque o caso só testava um lado do limite: entrar no dia em que a outra sai. Faltava sair no dia em que a outra entra e uma estadia inteira antes da outra.

Cenário acrescentado:

1. Já existe uma reserva do quarto 101 de 10/03/2030 a 12/03/2030.
2. Ana reserva o 101 de 08/03/2030 a 10/03/2030 (sai no dia em que a outra entra).
3. Ana reserva o 101 de 05/03/2030 a 07/03/2030 (antes de tudo).

O que confere. As duas reservas são aceitas e passam a existir 3 reservas.

Como mata o mutante. S1 ('<' virou '<=') e S2 ('<' virou 'is not') acusam conflito na estadia encostada de 08 a 10/03; S3 ('<' virou '!=') acusa conflito na estadia de 05 a 07/03, que nem encosta. Os mutantes recusam reservas válidas e o teste falha: mortos.

Código (`tests/test_03_mutacao.py`, linha 31):

```
@mutacao
@pytest.mark.ce("CE-15", "CE-21")
def test_CT_003_ampliacao_estadias_antes_da_reserva_existente(client, baseline):
    # Mata _periodos_conflitam, ReplaceComparisonOperator_Lt_LtE / Lt_IsNot / Lt_NotEq (ocorrência 1):
    # o caso só testava o lado "entrar no dia em que a outra sai"; faltava "sair no dia em que a
    # outra entra" (08 a 10/03) e uma estadia inteira antes da outra (05 a 07/03).
    reserva_existente(baseline["ana"], entrada="10/03/2030", saida="12/03/2030")
    login(client)
    assert path(reservar(client, entrada="08/03/2030", saida="10/03/2030")) == "/rooms"
    assert path(reservar(client, entrada="05/03/2030", saida="07/03/2030")) == "/rooms"
    assert Reservations.query.count() == 3
```

CT-012 ampliado na etapa de mutação: A ocupação de outra reserva não é herdada pelo quarto

Caso original: Quarto ocupado no período é recusado

Por que foi ampliado. O mutante S4 sobreviveu: na consulta de quartos ocupados, a junção 'Booked.brid == Reservations.rid' virou '>='. Faltava um cenário em que a reserva mais nova é do quarto pedido e a mais antiga é de outro quarto.

Cenário acrescentado:

1. Ana tem o 102 reservado de 10/03/2030 a 12/03/2030 (reserva nº 1).
2. Bruno tem o 101 reservado de 20/03/2030 a 22/03/2030 (reserva nº 2).
3. Ana reserva o 101 de 10/03/2030 a 12/03/2030.

O que confere. O sistema aceita (o 101 só está ocupado em 20/03) e passam a existir 3 reservas.

Como mata o mutante. Com '>=', o vínculo do 101 (reserva 2) também é ligado à reserva 1 e 'herda' as datas de 10/03. O mutante acha o 101 ocupado e recusa: o teste falha e o mutante morre.

Código (tests/test_03_mutacao.py, linha 44):

```
@mutacao
@pytest.mark.ce("CE-15")
def test_CT_012_ampliacao_ocupacao_de_outra_reserva_nao_e_herdada(client, baseline):
    # Mata _quartos_ocupados, ReplaceComparisonOperator_Eq_GtE: junção Booked.brid >= Reservations.rid.
    # O 101 (reserva 2, em 20/03) não pode herdar as datas da reserva 1 (quarto 102, em 10/03).
    reserva_existente(baseline["ana"], quartos=(102,), entrada="10/03/2030", saida="12/03/2030")
    reserva_existente(baseline["bruno"], quartos=(101,), entrada="20/03/2030", saida="22/03/2030")
    login(client)
    assert path(reservar(client, quartos="101", entrada="10/03/2030", saida="12/03/2030")) == "/rooms"
    assert Reservations.query.count() == 3
```

CT-015 ampliado na etapa de mutação: Saída antes da entrada também é recusada

Caso original: Saída igual à entrada é recusada

Por que foi ampliado. O mutante S6 sobreviveu: em reserve, 'saída <= entrada' virou 'saída == entrada'. O caso usava um único exemplo da classe CE-22 (0 noites), que continua recusado no mutante. Faltava o segundo exemplo da classe: saída antes da entrada.

Cenário acrescentado:

1. Ana está logada.
2. Ela tenta reservar com entrada em 10/03/2030 e saída em 09/03/2030.

O que confere. O sistema recusa e não grava nada.

Como mata o mutante. No mutante, 09/03 == 10/03 é falso, a validação deixa passar e o sistema grava uma reserva de -1 noite com custo negativo. O teste espera recusa: mutante morto.

Código (tests/test_03_mutacao.py, linha 56):

```
@mutacao
@pytest.mark.ce("CE-22")
def test_CT_015_ampliacao_saida_antes_da_entrada_e_recusada(client):
    # Mata reserve, ReplaceComparisonOperator_LtE_Eq (ocorrência 11): "d2 == d1" só recusa 0 noites.
    # O caso usava um único representante da classe CE-22 (saída igual à entrada).
    login(client)
    assert path(reservar(client, entrada="10/03/2030", saida="09/03/2030")) == "/reserve"
    assert Reservations.query.count() == 0
```

8. Perguntas prováveis e respostas

Por que só 15 casos funcionais?

Porque a técnica pede um caso por classe inválida (11) e permite cobrir todas as classes válidas com poucos casos. Os valores-limite foram embutidos nesses mesmos casos (o caso inválido usa o valor logo fora do limite e o válido usa o valor no limite). Com isso, 15 casos revelaram 9 dos 10 defeitos do recorte.

Por que a consulta de disponibilidade ficou de fora do recorte?

Ela tem formulário e regras próprias. Para manter o recorte enxuto, concentrei os três requisitos na reserva (reservar, data de entrada, data de saída). A consulta e o cancelamento ficaram como requisitos secundários, com testes complementares que também revelaram e confirmaram defeitos (DEF-08 a DEF-14, DEF-16 e DEF-17).

O que é o xfail estrito e por que usar?

É o jeito de provar o defeito no código original: o mesmo teste precisa FALHAR no original e PASSAR no corrigido. Se o defeito deixasse de existir sem correção, o teste avisaria.

Por que a cobertura de desvios ficou em 97,1% no original?

Falta só o desvio 230→229, que é inviável: os IDs das reservas são crescentes, então a condição nunca é falsa. Todos os desvios viáveis foram cobertos. No código corrigido esse laço sumiu e a cobertura é 100%.

Se a cobertura era 100%, por que ainda havia mutantes vivos?

Porque cobertura mede o que foi EXECUTADO, não o que foi VERIFICADO. Um teste pode passar por uma linha sem conferir o resultado dela no ponto exato. A mutação mostrou 7 fraquezas: limite testado de um lado só, um único exemplo de classe inválida, dados parecidos demais (quartos < 256) e ordem fixa de criação das reservas.

Por que corrigir os defeitos antes da mutação?

A mutação precisa de uma suíte que passe 100% no programa. Com 10 testes falhando no original, não daria para saber se uma falha veio da mutação ou do defeito que já existia.

O que é um mutante equivalente? Dê um exemplo.

É uma mudança que não altera o comportamento. Ex.: o custo da reserva é criado como 0 e depois substituído por `cal_cost` antes de salvar; trocar esse 0 por 1 não muda nada. Os 7 equivalentes estão justificados um a um no relatório (S8 a S14).

Qual o defeito mais grave?

DEF-01: a condição de conflito de datas é uma tautologia (sempre verdadeira). Depois da primeira reserva, o quarto nunca mais podia ser reservado. Foi revelado pelo valor limite "entrar no dia em que a outra estadia sai" (CT-003).

O que significa defeito mascarado?

DEF-15 só aparece depois que o DEF-01 é corrigido. Com a tautologia, tudo era recusado; mesmo corrigindo só a condição, o laço continuava juntando o quarto de uma reserva com as datas de outra (ampliação estrutural do CT-012).

Por que vocês não criaram casos novos nas etapas 2 e 3?

Porque os 15 casos funcionais já percorriam quase todo o código. Na estrutural, reaproveitei 5 deles (um por caminho principal da reserva) e só AMPLIEI 3 com o cenário que faltava para a cobertura. Na mutação, os 15 rodaram contra cada mutante e ampliei 4 para matar os sobreviventes. A ampliação é outra função `pytest` com o mesmo número de caso, então a rastreabilidade continua a mesma.

Por que as datas são de 2030?

Para serem datas reais e fixas que continuam no futuro (a reserva não aceita datas passadas). Só os casos de 'hoje' e 'ontem' usam a data do dia da execução, porque testam exatamente essa fronteira.

Como foi calculado o escore de mutação?

Inicial: $127 \div 141 = 90,1\%$. Final: $134 \div 141 = 95,0\%$. Ajustado (sem os 7 equivalentes): $134 \div (141 - 7) = 100\%$.

9. Roteiro da demonstração no V&V TestLab

Antes: `npm run dev` e importar `output/hotel-vvtestlab-backup.json` em Dados e exportação.

1. Visão geral: "Os mesmos 15 casos nas três etapas, 10 defeitos no recorte, 100% de comandos e desvios, 95% de escore de mutação (100% sem equivalentes)."
2. Teste funcional → abra "Como a etapa funcional foi feita" e explique as classes e os limites. Filtre "Resultado no original = Falhou" e abra o CT-003 (DEF-01) ou o CT-009 (zero hóspedes). Mostre: Em uma frase → cenário → o que aconteceu no original → tabela → assertivas → código.
3. Teste estrutural → explique que 5 funcionais foram reaproveitados. Abra o CT-012: mostre o grafo de reserve com o caminho em verde, clique nos nós para ver o código e conte a ampliação que revelou o defeito mascarado (DEF-15).
4. Teste de mutação → explique o cálculo (90,1% → 95,0% → 100% ajustado). Abra o CT-002: a ampliação reserva o hotel inteiro, incluindo o quarto 301, e mata dois mutantes (`==` virou `is`; `<=` virou `<`).